

programmingTest1 Lernzettel

Disclaimer!

Diese Zusammenfassung zielt ab, auf den 1. Programmieretest vorzubereiten. Man sollte diese Datei nicht zum Erlernen verwenden sondern eher als Nachschlagewerk, da einiges weggelassen wurde, was den meisten eh schon klar ist (Beispielsweise Dinge die in Java und C99 ident sind). Vieles in diesem Dokument wurde mit LLMs erstellt und könnte daher Fehler enthalten.

Inhalt

- [1. Unix introduction](#)
- [2. C introduction](#)
- [3. C Programming](#)
- [4. C Pitfalls](#)
- [5. Development in C](#)
- [6. myGrep + 3Coloring Handout](#)
- [7. myGrep Code-Walkthrough](#)
- [8. 3Coloring Code-Walkthrough](#)

1. Unix introduction

Info

Die Unix Introduction ist nicht sehr Testrelevant und ist demnach hier sehr gekürzt und nur auf C bezogen.

Warum C?

- **Einfache und effektive Sprache:**
 - **Konstrukte** bilden **klar auf Maschineninstruktionen** ab.
 - **Einfache Compiler-Implementierung.**
 - Erzeugt **sehr effizienten Code** für viele **Plattformen**.
- **Systemnähe & Einfluss:**
 - Hat viele Sprachen beeinflusst (*C* + *+*, *Java*, ...).
 - *UNIX*-Schnittstelle (*System – Calls*) wurde **für C** entworfen.
 - **Keine Abstraktionen: Volle Sicht** auf Prozesse.
 - Z.B. **Manuelle Speicherverwaltung** = Arbeiten mit **rohen (virtuellen) Speicheradressen**.

Warum nicht (andere Sprachen)?

Hochsprachen (Python, Ruby, ...)

- Machen **Performance-Kompromisse**.
- Sind oft **in C implementiert** (z.B. *CPython*).
- Erfolg beruht auch auf **C-Schnittstellen** (*ctypes*).

Moderne Systemprogrammiersprachen (Rust, Go, Zig, ...)

- Zeigen **Potenzial** (z.B. *Rust* im *Linux*-Kernel).
- Man versteht deren Design-Entscheidungen **besser nach dem Erlernen von C**.
- Könnten C's **Dominanz langfristig bedrohen**.
- **C bleibt auf absehbare Zeit relevant**.

Merke

C's Stärke liegt in der **Effizienz** und der **Hardware-Nähe**, was es zur Basis vieler anderer Systeme und Sprachen macht.

Prozesse (Processes)

Ein **Prozess** ist die Darstellung eines **laufenden Programms** durch das Betriebssystem (*OS*).

- **Bestandteile eines Prozesses:**
 - **Programmcode** (die auszuführenden Anweisungen).
 - **Programm-Daten** (Variablen, usw.).
 - **Kontext** (Zustand, Program-Counter, Prozessorregister).
- **Ausführung und Identifikation:**
 - *UNIX*-Systeme führen viele Prozesse **nebenläufig (concurrently)** aus.
 - Jeder Prozess hat einen **Identifikator (*PID*)**.
- **Hierarchie:**
 - Jeder Prozess, außer *PID* 1 (**init**), hat einen **Elternprozess (*parent*)**.
 - Neue Prozesse sind **Kindprozesse (*children*)** des Prozesses, der sie gestartet hat (z.B. Prozesse, die von einer Shell gestartet werden, sind Kinder der Shell).
- **Beendigung:**
 - Prozesse liefern bei ihrer Beendigung einen **Exit-Status** (auch *exit value, return code* genannt).
 - **0 = Erfolg (*success*)**.
 - Andere ganze Zahlen = **Fehler (*failure*)**.
- **Kommandozeilen-Tools:**
 - `ps`: Zeigt die aktuell laufenden Prozesse.
 - `pstree`: Zeigt die **Prozesshierarchie** (Eltern/Kinder).

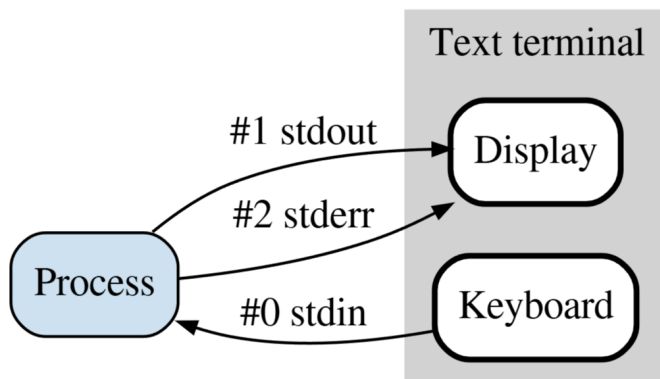
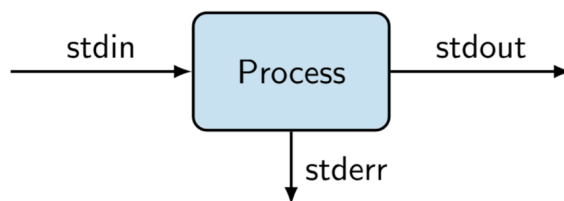
Standard Input/Output (STDIO)

In *UNIX* ist "alles eine Datei" (eine Abfolge von Bytes). Prozesse verwenden **Datei-Deskriptoren** (*file descriptors*) zur Verfolgung offener Dateien.

🔗 Wichtig

Nicht alle Dateien sind auf der Festplatte gespeichert (z.B. Netzwerk-Sockets oder Terminals sind auch "Dateien").

- **Drei spezielle Datei-Deskriptoren (*fd*)** existieren bei jedem Prozessesstart:
 1. **Standard Input** (*stdin*, *fd* 0): Für die Eingabe von Daten.
 2. **Standard Output** (*stdout*, *fd* 1): Für die normale Ausgabe.
 3. **Standard Error** (*stderr*, *fd* 2): Für Fehlermeldungen.



Wenn ein Prozess in einer Shell gestartet wird, ist das Standard *I/O* mit dem **Terminal** verbunden (Eingabe über Tastatur, Ausgabe/Fehler auf dem Bildschirm).

📄 Info

`stdin`, `stdout` und `stderr` werden meist gemeinsam angezeigt:

```

user@host:~$ ls *.txt doesNotExist.tar
file.txt
ls: cannot access 'doesNotExist.tar': No such file or directory

(shell prompt, stdout of shell)
(user input, stdin of shell)
(stdout of ls)
(stderr of ls)
  
```

Navigieren im Dateisystem

Wichtige **Kommandozeilen-Befehle** zur Verwaltung und Navigation im Dateisystem:

- `cd`: **c**hange **d**irectory (Verzeichnis wechseln).
- `pwd`: **p**rint **w**orking **d**irectory (aktuelles Verzeichnis ausgeben).
- `ls`: **l**ist **d**irectory **c**ontents (Inhalt eines Verzeichnisses auflisten).
 - *Beispiel:* `$ ls -l` zeigt Details zur Datei `textfile` (Berechtigungen, Eigentümer, Größe, Datum).
- `mkdir`: **m**ake **d**irectories (neue Verzeichnisse erstellen).
 - *Beispiel:* `$ mkdir test`
- `rmdir`: **r**emove **d**irectories (Verzeichnisse löschen).

Spezielle Pfadangaben

- `.`: Steht für das **aktuelle Verzeichnis** (*current directory*).
- `..`: Steht für das **Elternverzeichnis** (*parent directory*).

Pfadtypen

1. Absolute Pfade:

- Beginnen immer mit einem **Slash** (`/`) vom **Root-Verzeichnis** aus.
- *Beispiel:* `$ cat /etc/passwd`

2. Relative Pfade:

- **Beginnen nicht mit einem Slash** (`/`).
- Beziehen sich immer auf das **aktuelle Verzeichnis**.
- *Beispiel:* `$ cat ../tmpfile` (Greift auf eine Datei im Elternverzeichnis zu).

Wildcards (Platzhalter)

Wildcards sind Muster (*pattern matching*), um **mehrere Dateinamen** mit einem einzigen Ausdruck zu spezifizieren.

- **Platzhalter und ihre Bedeutung:**
 - `*`: Steht für **null oder mehr Zeichen**.
 - `?`: Steht für **genau ein einziges Zeichen**.
 - `[xyz]`: Steht für **eines der Zeichen** 'x', 'y' oder 'z'.
 - `[a-i]`: Steht für **ein Zeichen im Bereich** von 'a' bis 'i'.

Globbing

Der **Shell-Vorgang**, bei dem sie Wildcards interpretiert und diese in eine **Liste von Dateinamen** auflöst (*expands*), wird als **Globbing** bezeichnet.

- **Wildcards als normale Zeichen verwenden:**

- Um einen Platzhalter (*, ?, etc.) als normales Zeichen und nicht als Wildcard zu verwenden, muss er entweder **in Anführungszeichen** gesetzt oder mit einem **vorangestellten Backslash (\)** maskiert werden.

2. C introduction

Fakten

- Wir verwenden **C99**
- **Imperativ:**
 - Die Ausführung basiert darauf, den **Zustand** (Werte von **Variablen**) durch **Anweisungen** (*instructions*) zu ändern.
- **Prozedural:**
 - Anweisungen sind in **Prozeduren (Funktionen)** gruppiert.
 - Funktionen können **Argumente** entgegennehmen und **Rückgabewerte** liefern.
- **Statisch Typisiert (Statically Typed):**
 - Jede Variable hat einen **festen Typ** (z.B. *integer, floating point, character*), der sich **während der Laufzeit nicht ändert**.
 - Funktionen haben ebenfalls einen definierten **Rückgabebetyp (return type)**.
- **Kompiliert (Compiled):**
 - Programme werden durch einen **Compiler** in **Maschinencode** übersetzt, bevor sie ausgeführt werden.
- **Vorverarbeitet (Preprocessed):**
 - Der Standard definiert einen **Präprozessor**, der den **Quellcode vor der eigentlichen Kompilierung verändert**.
 - Wird genutzt, um z.B. **anderen Code einzubinden** oder **Makros** (textuelle Ersetzungen) zu definieren.
 - Constants werden eingebunden
 - `#include` code wird in den source code reinkopiert

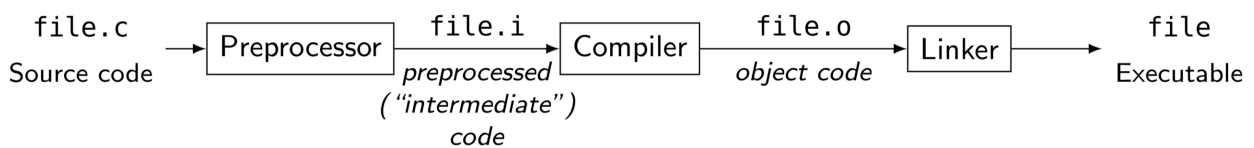
Hello, C World

```
#include <stdio.h> // Loads contents of stdio.h which is part of standard
lib
                        // (instruction to preprocessor)

int main(void) {        //Defines main function
    printf("Hello World\n") // prints to stout
    return 0;           //Exit code of programm
}
```

Compilation

Using GCC: `$ gcc -o prog prog.c`



Signiert und Unsígníert (Signed vs. Unsigned)

1. Signed (Mit Vorzeichen)

- **Zahlenbereich:** Speichert **positive, negative Zahlen** und **Null**.
- **Interpretation:** Das **höchstwertige Bit (MSB)** ist das **Vorzeichen-Bit**.
- **Wertebereich:** Reduziert, da ein Bit für das Vorzeichen reserviert ist.
- **Standard:** `int` und `char` sind **standardmäßig signed**.

2. Unsigned (Vorzeichenlos)

- **Zahlenbereich:** Speichert **nur positive Zahlen** und **Null**.
- **Interpretation:** **Alle Bits** werden zur Speicherung des **Betrags** der Zahl verwendet.
- **Wertebereich:** Der gesamte Speicherplatz wird für **größere positive Werte** genutzt (doppelt so groß wie bei `signed`).
- **Typische Nutzung:** Für Zähler oder Indizes.

Kurzfassung

`signed` erlaubt negative Zahlen. `unsigned` maximiert den positiven Wertebereich.

Bitweise Operatoren (Bitwise Operators)

Bitweise Operatoren arbeiten **Bit für Bit** (*bit by bit*) auf ihren Operanden (Ganzzahlen).

Operator	Erklärung
<code>x & y</code>	AND
<code>**`x</code>	<code>y`**</code>
<code>x ^ y</code>	Exklusives OR (XOR)
<code>~x</code>	Bitweises Komplement (invertiert alle Bits).
<code>x >> n</code>	Shift Right um <i>n</i> Positionen.
<code>x << n</code>	Shift Left um <i>n</i> Positionen.

- **Ergänzung zu `~x`**: Das bitweise Komplement `~ 0` ist `1` (alle Bits auf `1` gesetzt) und umgekehrt.

sizeof Operator

Der Operator `sizeof` wird verwendet, um den **Speicherbedarf** (*memory needed*) für einen bestimmten **Typ** oder eine **Variable** in **Bytes** abzurufen.

- **Syntax**: `sizeof` ist ein **Operator** (nicht zwingend eine Funktion); daher kann die Syntax "strange" wirken (z.B. `sizeof i` statt `sizeof(i)`).

Konstanten (Constants)

Es gibt zwei Hauptarten, Konstanten zu definieren:

1. C Konstanten (mittels `const`)

- `const` definiert eine **read-only variable** (eine Variable, deren Wert nach der Initialisierung nicht mehr geändert werden kann).
 - **Beispiel**: `const int i = 23;`

2. Präprozessor-Makros (mittels `#define`)

- `#define` wird **durch den Präprozessor ersetzt**.
 - Der definierte Name wird **textuell** durch seinen Wert ersetzt, **bevor der Compiler den Code überhaupt sieht** (*will not be seen by the compiler at all!*).
 - **Beispiel**: `#define MYCONST 23`

Wahrheitsgehalt

In *C* wird der Wahrheitsgehalt von numerischen Werten wie folgt interpretiert:

- 0 (Null) gilt als **falsch** (*false*).
 - **Alles andere** (jede Zahl $\neq 0$) gilt als **wahr** (*true*).
-

Strings in C

In C sind **Strings** grundlegend **Arrays von Zeichen** (*char*).

- **Terminierung:** Strings werden zwingend mit dem **Null-Terminator** (`\0`) abgeschlossen.
 - **Grund:** Dies ist **essenziell**, damit Funktionen (wie z.B. `printf` mit `%s`) das **Ende des Strings erkennen**.

Deklaration und Initialisierung

1. Initialisierung als String-Literal:

- `char string[] = "hello, world";`
- Bei String-Literalen (`"..."`) wird der **Null-Terminator** (`\0`) **automatisch angefügt**.

2. Manuelle Initialisierung:

- Strings können als *char*-Arrays definiert werden, wobei der **Null-Terminator** (`\0`) **explizit** an der letzten Position gesetzt werden muss.

String-Manipulation

- **Ausgabe:**
 - `printf("%s\n", s);` gibt den gesamten String bis zum ersten `\0` aus.
 - **Dynamische Verkürzung:**
 - Strings können verkürzt werden, indem der **Null-Terminator an einer früheren Stelle** im Array platziert wird.
 - **Beispiel:** Das Setzen von `s[3] = '\0'` würde den String auf die ersten drei Zeichen (`"heL"`) kürzen.
-

Schlüsselwörter zur Variablenkontrolle

1. `volatile`

Das Schlüsselwort `volatile` signalisiert dem Compiler, dass eine Variable **durch die Umgebung außerhalb des Programms** (*outside of the program*) geändert werden kann.

- **Bedeutung:** Die Variable kann sich unvorhergesehen ändern (z.B. durch Hardware oder einen anderen Thread).
- **Anwendungsfall:** Ist **wichtig für die Hardware-Programmierung** (z.B. eine Variable, die auf einen **Sensor** gemappt ist).

2. `extern`

- **Zweck: Deklariert Variablen**, die **in einer anderen Datei definiert** (*defined in another file*) sind.
- **Effekt:** Informiert den Linker, dass die tatsächliche Definition und Speicherzuweisung woanders stattfindet.

3. **static**

Das Schlüsselwort `static` hat zwei Hauptbedeutungen, je nachdem, wo es verwendet wird:

A) Gültigkeitsdauer (Innerhalb einer Funktion)

- **Speicherzuweisung:** Gibt der Variablen einen **festen Speicherplatz** (*fixed memory space*).
- **Zustand:** Ihr **Zustand bleibt erhalten** (*its state remains*) zwischen Funktionsaufrufen (im Gegensatz zu lokalen Variablen auf dem Stack).

B) Sichtbarkeit (Global oder in Datei/Block)

- **Zugriffskontrolle:** Eine `static` Variable kann **nicht von einem Block oder einer Datei außerhalb** des aktuellen Gültigkeitsbereichs zugegriffen werden (*cannot be accessed from an outside block or file*).
- **Implikation:** Wird eine globale Variable als `static` deklariert, ist sie nur innerhalb der Datei sichtbar, in der sie definiert wurde (Dateisichtbarkeit).

Pointers

Ein **Pointer** ist eine **Variable**, die die **Adresse (Speicherstelle)** einer anderen Variablen speichert.

- **Zweck:** Ermöglicht den **Zugriff auf den Wert einer Variablen, ohne deren Namen** verwenden zu müssen.
- **Häufige Anwendung in C:**
 - Zugriffe auf **Arrays** erfolgen über Pointer.
 - Wenn Speicher vom System angefordert wird, erhält man einen Pointer.
 - Funktionen können einen Pointer auf einen Speicherbereich erhalten, mit dem sie arbeiten sollen.

Deklaration

- **Syntax:** Pointer werden deklariert mit `Typ *Name`.
- **Speichergröße:** Der reservierte Speicherplatz für den Pointer selbst entspricht **nicht** der Größe des referenzierten Typs, sondern der Größe einer **Speicheradresse** (meist 4 oder 8 Bytes).
- **Operatoren:**
 - **Dereferenzierungsoperator** (`*`): Greift auf den **Wert** zu, auf den der Pointer zeigt.
 - **Adressoperator** (`&`): Liefert die **Adresse** einer Variablen.
- **Wichtig bei Deklaration:** Das Sternchen (`*`) bindet stärker an den Variablennamen. `int* a, b;` deklariert *a* als Pointer und *b* als normalen *int*. Um beide als Pointer zu deklarieren, muss *b* ebenfalls ein Sternchen erhalten: `int *a, *b;`.

Simple Pointer Arithmetic

Bei der Pointerarithmetik wird die Speicheradresse basierend auf dem Typ des Pointers verschoben.

- **Definition:**
 - Ein Pointer wird auf die Adresse des ersten Elements gesetzt: `int *p = ar[0];`.
 - Alternativ: `p = ar;` (der Array-Name `ar` ist **kein Pointer, sondern nur die Startadresse** des Arrays).
 - **Addition/Subtraktion:**
 - Die Operation `p + 1` verschiebt den Pointer zur **Adresse des nächsten Elements** im Array, **unabhängig von der Byte-Größe** des Typs.
 - **Beispiel:** Wenn ein `int` 4 Bytes groß ist, erhöht `p + 1` die Adresse um 4 Bytes.
 - Das Dereferenzieren `*p` liefert den Wert am aktuellen Speicherort.
-

Risiken von Pointern

Pointerarithmetik kann **riskant** werden, **wenn man nicht sorgfältig arbeitet** (*if you do not work with care*).

- **Gefährliche Arithmetik:**
 - Das Hinzufügen einer großen, willkürlichen Zahl (`p += 23;`) kann zu Problemen führen, da man in Speicherbereiche außerhalb des zugewiesenen Arrays gelangt.
 - **Null-Pointer-Dereferenzierung:**
 - Das Dereferenzieren eines **Null-Pointers** (`p = NULL`) war eines der häufigsten Sicherheitsprobleme. Dies führt zu einem Programmabsturz oder undefiniertem Verhalten.
-

Undefined Behaviour

Das **Ergebnis einiger Operationen** wird durch den C-Standard **explizit als undefiniert** (*undefined*) festgelegt.

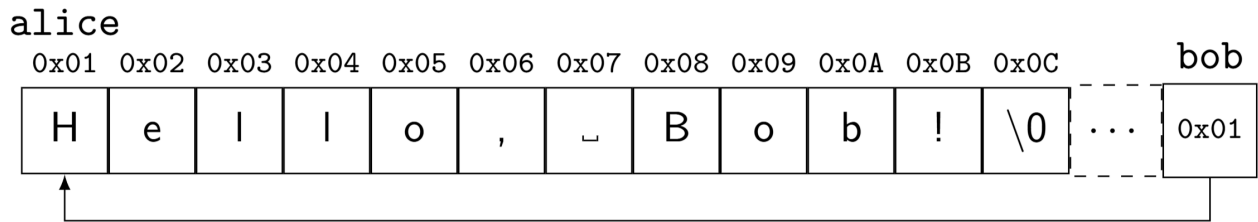
- **Beispiele für undefiniertes Verhalten:**
 - **Division durch null.**
 - **Array-Zugriff außerhalb der Grenzen** (*Out of bounds array access*).
 - Ausdrücke mit "seltsamen" Seiteneffekten auf den Zustand, z.B. `y = (++x) + (++x)`.
- **Konsequenz:**
 - Der **Compiler** darf diese Fälle **behandeln, wie er möchte**
 - Das **Programm könnte abstürzen**

- Oder es **läuft weiter, liefert aber falsche Ergebnisse** (falsch aus Sicht des Programmierers).
- **Tipp:**
 - **Alle Warnungen aktivieren** (wie das geht, wird später gezeigt). Diese helfen, undefiniertes Verhalten frühzeitig zu erkennen.

3. C Programming

Pointer Basics

Pointer speichern Speicheradressen. Sie sind fundamental für die Arbeit mit Arrays, Strings und dynamischem Speicher in C.



- **String Literals vs. Arrays:**

- `char alice[] = "Hello";` $\implies$ Speicher liegt im Stack/Data Segment, ist veränderbar.
- `char *bob = "Hello";` $\implies$ Pointer zeigt auf ein String Literal (oft Read-Only Memory!).
- **Achtung:** `bob[0] = 'R';` kann zu undefined behavior (Absturz) führen, auch wenn der Compiler nur eine Warnung gibt.

- **Pointer Arithmetic:**

- `bob += 7;` bewegt den Pointer um 7 *Elemente* weiter (nicht unbedingt 7 Bytes, hängt vom Datentyp ab, bei `char` sind es 1 Byte).

Casting & Endianness

Casting zwingt den Compiler, einen Wert als einen anderen Typ zu behandeln.

- **Syntax:** `(type) expression`
- **Floats zu Int:** Schneidet Nachkommastellen ab (`3.14` $\implies$ `3`).
- **Int zu Int:**
 - Larger type: Wert bleibt erhalten.
 - Smaller type: Höhere Bits werden abgeschnitten.
 - Signed/Unsigned: Kann den Wert verändern.

Endianness (Byte Order)

Wenn man Pointer castet, ist die Byte-Reihenfolge der Maschine wichtig.

Beispiel: `int x = 2063` (Hex `0x80F`).

Little Endian:

lowest byte first

x

0xf	0x80	0x0	0x0
-----	------	-----	-----

Big Endian:

highest byte first

x

0	0	0x80	0xf
---	---	------	-----

- **Little Endian** (Intel, AMD, Apple Silicon): Niederwertigstes Byte zuerst (`0xf`, `0x8`, `0x0`, `0x0`).
- **Big Endian** (z.B. Netzwerke, ältere MIPS): Höherwertigstes Byte zuerst.

Structs

`struct` fasst Variablen zu einer logischen Einheit zusammen (z.B. ein User-Account). Speicher liegt als zusammenhängender Block vor.

```
struct account {
    char username[32];
    char password[32];
    unsigned int uid;
};
```

- **Initialisierung:**
 - C89: `struct account user = {"alice", "pass", 1};` (Reihenfolge wichtig!)
 - C99 (Designated Initializers): `struct account user = {.uid=1, .username="alice"};` (Besser lesbar).
- **Zugriff:** Mit dem Punkt-Operator `.` (z.B. `user.uid`).

Alignment & Padding

Der Compiler fügt oft Füllbytes (Padding) ein, damit Daten an Adressen liegen, die ein Vielfaches ihrer Größe sind (z.B. Int an durch 4 teilbaren Adressen).

- **Konsequenz:** `sizeof(struct)` ist oft größer als die Summe der Einzelteile.
- Die Reihenfolge der Variablen im Struct beeinflusst die Gesamtgröße!

Pointer auf Structs

Zugriff auf Felder über einen Pointer:

1. Dereferenzieren und Punkt: `(*p).uid`
2. **Arrow-Operator (Syntactic Sugar):** `p->uid` (Standard, besser lesbar).

Typedef

Erstellt Alias-Namen für Datentypen. Besonders nützlich, um das ständige Schreiben von `struct` zu vermeiden.

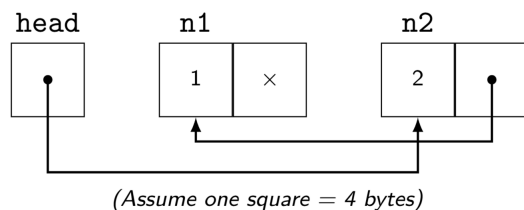
```
// Definition und Typedef in einem
typedef struct account {
    char username[32];
    unsigned int uid;
} account_t;

account_t user1; // statt "struct account user1"
```

Linked Lists (Verkettete Listen)

Eine dynamische Datenstruktur, bei der jedes Element (Node) einen Pointer auf das nächste Element enthält.

```
typedef struct node {
    int data;
    struct node *next;
} node_t;
```



Funktionen & Parameterübergabe

Void

- `void`: Leerer Datentyp (kein Rückgabewert).
- `void*`: Pointer ohne Typinformation (generischer Pointer). Kann auf alles zeigen, aber keine Arithmetik oder Dereferenzierung ohne Cast erlaubt.

Call by Value vs. Call by Reference

1. Call by Value (Standard in C):

- Funktion arbeitet mit einer **Kopie** der Variable.
- Änderungen in der Funktion betreffen **nicht** die originale Variable.


```
void foo(int n) { n = 23; } // Original bleibt unverändert
```

2. Call by Reference (via Pointer):

- Man übergibt die Adresse der Variable (`&a`).
- Die Funktion hat einen Pointer auf das Original.
- Änderungen via Dereferenzierung (`*n = ...`) ändern das **Original**.
- **Wichtig:** Arrays werden in C *immer* implizit als Pointer auf das erste Element übergeben (effektiv Call by Reference).

```
void foo(int *n) { *n = 23; } // Original wird geändert
```

Const Pointer

`const` schützt vor versehentlichen Änderungen. Wichtig von rechts nach links zu lesen:

1. `const int *p`: Pointer auf einen konstanten Integer.
 - Wert ändern (`*p = 1`) $\implies$ **Verboten**.
 - Pointer umbiegen (`p = &b`) $\implies$ Erlaubt.
2. `int * const p`: Konstanter Pointer auf einen Integer.
 - Wert ändern (`*p = 1`) $\implies$ Erlaubt.
 - Pointer umbiegen (`p = &b`) $\implies$ **Verboten**.

Linked List Operations

Push (Element vorne anfügen)

Um den `head`-Pointer der Liste innerhalb einer Funktion zu ändern, muss man einen Pointer auf den Head-Pointer übergeben (`node_t **`).

```
void push(node_t **hd, const int data) {
    node_t *newNode = malloc(sizeof(node_t)); // Speicher reservieren
    // Fehlerbehandlung für malloc fehlt hier (wichtig in Praxis!)
    newNode->data = data;
    newNode->next = *hd; // Neuer Node zeigt auf alten Head
    *hd = newNode;      // Head zeigt auf neuen Node
}
```

Pop (Element entfernen)

Entfernt das erste Element und gibt Daten frei.

- `free(ptr)` gibt den Speicher frei, setzt den Pointer aber nicht auf NULL (Dangling Pointer Gefahr!).
- Zugriff nach `free` $\implies$ Undefined Behavior.

```
int pop(node_t **hd) {
    if (*hd == NULL) return -1; // Liste leer
    node_t *tmp = *hd;          // Temporärer Pointer auf das zu löschende
    Element
    int data = tmp->data;
    *hd = tmp->next;             // Head weiterrücken
    free(tmp);                  // Alten Head freigeben
    return data;
}
```

Unions

Unions teilen sich denselben Speicherplatz für verschiedene Datentypen.

- `sizeof(union)` = Größe des *größten* Elements.
- Es kann immer nur ein Member gleichzeitig sinnvoll genutzt werden.

```
union number {
    char c;    // 1 Byte
    short s;   // 2 Byte
};
// s zu schreiben überschreibt c!
```

Tagged Unions

Kombination aus `struct`, `enum` (Tag) und `union`, um Polymorphismus zu simulieren. Das `enum` merkt sich, welcher Typ aktuell in der Union gespeichert ist.

```
1  typedef enum types {          9  typedef struct node {
2      A_FLOAT, A_INT           10      data_t data;
3  } types_t;                  11      struct node *next;
4  typedef struct data {        12  } node_t;
5      types_t type;           13  //...
6      union {                 14  node_t *head = NULL;
7          int i;               15  data_t d1 = { A_INT, 42 };
8          float f;             16  data_t d2 = { .type=A_FLOAT, .value.f=3.14 };
9      } value;                 17  push(&head, d1);
10 } data_t;                    18  push(&head, d2);
```

Function Pointers

Pointer können auch auf Programmcode (Funktionen) zeigen. Ermöglicht Callbacks oder dynamische Funktionsaufrufe.

- **Syntax:** `returnType (*name)(parameters)`
- Beispiel: `int (*f)(int, int) = &add;`

Modularisierung & Kompilierung

Code wird in Header (`.h`) und Source (`.c`) Files aufgeteilt.

Header Files (.h)

- Enthalten Prototypen (Deklarationen), Struct-Definitionen, Konstanten.
- **Keine** Funktionsimplementierungen.
- **Include Guards** (`#ifndef ... #define ... #endif`) verhindern doppeltes Einbinden.

Source Files (.c)

- Enthalten die eigentliche Implementierung.
- `static` vor einer Funktion macht sie "privat" für dieses File (nicht von außen sichtbar).

Kompilierung (Multi-file)

1. **Compile:** `.c` Dateien werden einzeln zu Object Files (`.o`) kompiliert.
 - `gcc -c linkedlist.c`
2. **Link:** Object Files werden zu einer Executable verbunden.
 - `gcc -o prog linkedlist.o prog.o`

Vorteil: Bei Änderungen muss nur die betroffene `.c` Datei neu kompiliert werden, nicht alles.

4. C Pitfalls

📄 Überblick

C ist ein mächtiges Werkzeug ("Carving Knife"), aber gefährlich, wenn man es nicht beherrscht. Fehler werden in drei Kategorien unterteilt:

1. **Syntactic**: Wie der Code geschrieben wird.
2. **Semantic**: Was der Code bedeutet/tut.
3. **Lexical**: Wie der Compiler Text in Tokens zerlegt.

Part I: Syntactic Pitfalls

Declarations

Das Prinzip in C lautet: "**Declaration follows use**". Die Deklaration zeigt, wie man durch Nutzung der Variable an den Basis-Datentyp kommt.

☰ Beispiele für komplexe Deklarationen

- `int *g(void);` $\Rightarrow$ Funktion `g`, die einen Pointer auf `int` zurückgibt.
- `int (*h)(void);` $\Rightarrow$ Pointer `h`, der auf eine Funktion zeigt, die `int` zurückgibt.
- `int *i, j;` $\Rightarrow$ `i` ist Pointer, `j` ist normaler `int`!

Bitwise vs. Logical Operators

Verwechslungsgefahr zwischen logischen Operatoren (für Bedingungen) und bitweisen Operatoren (für Bit-Manipulation)

- **Logisch**: `&&`, `||` (Short-circuit evaluation: bricht ab, sobald Ergebnis feststeht).
- **Bitwise**: `&`, `|` (Evaluert immer beide Operanden).

⚠️ Falle

`if (a & b)` kompiliert zwar, verhält sich aber anders als `if (a && b)`.

- `a & b` führt eine bitweise UND-Verknüpfung durch.

a:	0	0	0	0	0	0	1	0
b:	0	0	0	0	0	1	0	0
a & b:	0	0	0	0	0	0	0	0

Char Type

- Pointer `char` kann je nach Plattform `signed` oder `unsigned` sein.
- Für Text ist das egal, für kleine Integers muss man explizit `signed char` oder `unsigned char` schreiben.

Semicolons & Control Flow

PointerEin Strichpunkt `;` allein ist ein "Empty Statement"

⚡ Infinite Loop / Falsche Logik

```
if (x[i] > big); // <-- Das Semikolon beendet das IF sofort!
    big = x[i]; // Wird IMMER ausgeführt
```

Dangling Else

PointerEin `else` bezieht sich immer auf das **nächste** (unmittelbar davorstehende) `if`, es sei denn, Klammern `{}` erzwingen etwas anderes

🔗 Best Practice

Immer geschweifte Klammern `{}` verwenden, auch bei einzeiligen Blöcken.

Operator Precedence (Vorrangregeln)

PointerDie Reihenfolge, in der Ausdrücke ausgewertet werden, ist oft unintuitiv

Kritische Beispiele:

1. `if (flag & FLAG != 0)`
 - $\Rightarrow$ Wird interpretiert als `if (flag & (FLAG != 0))` weil `!=` höher bindet als `&`.
 - **Erwartet:** `if ((flag & FLAG) != 0)`
2. `r = h << 4 + 1`
 - $\Rightarrow$ Wird interpretiert als `r = h << (4 + 1)` weil `+` höher bindet als `<<`.
 - **Erwartet:** `r = (h << 4) + 1`

Operator	Explanation
()	Function call
[]	Array indexing
. ->	Structure element selection
++ --	Postfix increment/decrement (e.g., x++)
++ --	Prefix increment/decrement (e.g., ++x)
+ - ! ~	Unary operators: plus/minus/logical Negation/bitwise negation
(type)	Type casts
* &	Dereferencing/Address operator
sizeof	Determining the memory consumption in byte
* / %	Multiplication/division/modulo
+ -	Addition/Subtraction
<< >>	Bitwise shift left/right

< <= > >=	Relational operators: smaller than/smaller or equal/greater than/greater or equal
== !=	Relational operators: equal to/not equal to
&	bitwise AND
^	bitwise exclusive OR
	bitwise inclusive OR
&&	logical AND
	logical OR
?:	Ternary operator
=	Assignment
+= -= *=	Ass. with addition/subtraction/multiplication
/= %= &=	Ass. with division/modulo/bitwise AND
^= =	Ass. with exclusive OR/inclusive OR
<<= >>=	Ass. with bitwise shift left/right
,	Separation operator

Part II: Semantic Pitfalls

Implicit Initialization

- **Lokale Variablen:** Haben einen zufälligen Wert (Garbage).
- **Pointer/Globale/Statische Variablen:** Werden implizit mit `0` initialisiert
- **Moral:** Immer explizit initialisieren!

Pointers vs. Arrays

In C sind Arrays zusammenhängende Speicherblöcke. PointerArray-Zugriff `a[i]` ist nur eine Abkürzung für Pointer-Arithmetik `*(a+i)`

Array Decay

PointerWenn ein Array an eine Funktion übergeben wird, "zerfällt" (decays) es zu einem Pointer auf das erste Element (`&arr[0]`)

- Die Größe geht verloren! `sizeof(arr)` innerhalb der Funktion liefert die Größe des Pointers (z.B. 8 Byte), nicht des Arrays.

Unterschiede:

- `int a[] = {1,2};` $\Rightarrow$ `sizeof(a)` liefert Größe des gesamten Arrays.
- `int *p = a;` $\Rightarrow$ `sizeof(p)` liefert Größe des Pointers.

Order of Evaluation

Die Reihenfolge, in der Teile eines Ausdrucks ausgewertet werden, ist oft **undefiniert** (außer bei `&&`, `||`, `?:`, `,`)

⚡ Undefined Behavior Beispiel

```
while (i < n)
    y[i] = x[i++]; // DANGEROUS!
```

Es ist nicht definiert, ob `i` auf der linken Seite (`y[i]`) den alten oder den neuen Wert hat. Der Compiler entscheidet.

Dynamic Memory Management

Funktionen: `malloc`, `calloc`, `realloc`.

1. Pointer können `NULL` zurückgeben $\Rightarrow$ Immer prüfen!
2. `free(ptr)`:
 - `ptr` muss exakt die Adresse sein, die `malloc` zurückgab.
 - Pointer-Arithmetik vor `free` führt zum Crash, wenn man nicht zurückrechnet.

🚫 Segfault bei free

```
char *p = malloc(6);
p += 3; // Pointer verschieben
free(p); // UNDEFINED BEHAVIOR! Adresse ist falsch.
```

Strings & Buffer Overflows

Strings sind `char`-Arrays, terminiert mit `\0`.

Zuweisung:

- `char buf[] = "init";` (Kopie auf dem Stack, veränderbar).
- Pointer `char *buf = "init";` (Pointer auf Read-Only Memory, **nicht** veränderbar $\Rightarrow$ Segfault bei Schreibversuch)

Buffer Overflow:

Pointer `strcpy` prüft nicht, ob das Ziel groß genug ist

✓ Lösung: `strncpy`

```
strncpy(dst, src, MAX_LEN - 1);
dst[MAX_LEN - 1] = '\0'; // Manuell terminieren!
```

Pointer **Achtung:** `strncpy` null-terminiert **nicht** automatisch, wenn das Limit erreicht wird

Function	Explanation
<code>strlen(3)</code>	Length of the string (excl. terminating <code>'\0'</code>)
<code>strstr(3)</code>	Locates a substring
<code>strcpy(3)</code> , <code>strncpy(3)</code>	Copies a string
<code>strdup(3)</code> , <code>strndup(3)</code>	Duplicates a string
<code>strcmp(3)</code> , <code>strncmp(3)</code>	Compares strings
<code>strtol(3)</code> , <code>strtoll(3)</code>	String → integer conversion
<code>strtof(3)</code> , <code>strtod(3)</code> , <code>strtold(3)</code>	String → floating-point number conversion

Part III: Lexical Pitfalls

Multi Character Tokens

Der Lexer arbeitet stur von links nach rechts ("Greedy").

☰ Pointer vs. Division

```
y = x/*p;
```

Der Compiler sieht `/*` als Beginn eines Kommentars, nicht als `x` geteilt durch `*p`.

Lösung: Leerzeichen `y = x / *p;` oder Klammern `y = x / (*p);`.

Macros

Macros sind reine Textersetzung durch den Präprozessor.

Problem 1: Operator Precedence

```
#define SQR(x) x*x
```


- PointerAufruf: `SQR(a+b)` $\Rightarrow$ `a+b*a+b` (Punkt-vor-Strich Fehler!)
- PointerLösung: Klammern nutzen `#define SQR(x) ((x)*(x))`

Problem 2: Side Effects

- Aufruf: `SQR(a++)` $\Rightarrow$ `((a++)*(a++))`
- Pointer `a` wird zweimal inkrementiert $\Rightarrow$ Undefined Behavior

Problem 3: Multi-Statement Macros

Wenn ein Makro mehrere Befehle enthält, bricht es in `if/else` Strukturen ohne Klammern.

✓ Das do-while(0) Idiom

Um Makros sicher zu machen, verpackt man sie in einen `do-while(0)` Block. Dieser verhält sich syntaktisch wie ein einzelnes Statement.

```
#define CMDS \  
do { \  
    a = b; \  
    c = d; \  
} while(0)
```

5. Development in C

1. Build Automation mit **make**

Beim Kompilieren von Projekten mit mehreren Dateien ist es ineffizient, immer *alles* neu zu kompilieren (`gcc -o hello main.c hello.c`), wenn sich nur eine Datei geändert hat.

Der Prozess (Single vs. Multi-File)

1. **Compiler:** Erstellt Object Files (`*.o`) aus Source Files (`*.c`).
 - `gcc -c -o main.o main.c`
2. **Linker:** Verbindet Object Files zur Executable.
 - `gcc -o hello main.o hello.o`

✓ Vorteil von Make

Inkrementelle Builds: Das Betriebssystem speichert Änderungszeitstempel. `make` kompiliert nur die Dateien neu, deren Quellcode neuer ist als das zugehörige Object File. Das spart bei großen Projekten extrem viel Zeit.

Makefile Syntax

Ein Makefile besteht aus **Regeln (Rules)**.

```
target: dependencies
    recipe
```

- **Target (Ziel):** Die Datei, die erstellt werden soll (z.B. `hello` oder `main.o`).
- **Dependencies (Abhängigkeiten):** Dateien, die benötigt werden (z.B. `main.c`, `header.h`).
- **Recipe (Rezept):** Die Shell-Befehle, um das Target zu erstellen.

⚡ Wichtig: Indentierung

Befehle im Recipe müssen mit einem TAB eingerückt werden. Leerzeichen funktionieren nicht!

Beispiel

```
# Variablen definieren
CC = gcc
CFLAGS = -std=c99 -Wall

# Erstes Target wird standardmäßig ausgeführt
hello: main.o hello.o
    $(CC) $(CFLAGS) -o hello main.o hello.o

# Pattern Rule: Gilt für alle .o Dateien, die von .c abhängen
%.o: %.c
    $(CC) $(CFLAGS) -c -o $@ $<

# Explizite Abhängigkeiten für Header-Files
main.o: main.c hello.h
hello.o: hello.c hello.h
```

Automatische Variablen & Patterns

Um Schreibarbeit zu sparen, nutzt man Platzhalter:

Variable	Bedeutung
\$@	Name des Targets (z.B. <code>hello.o</code>)
\$<	Die erste Dependency (z.B. <code>hello.c</code>)
\$^	Alle Dependencies (z.B. <code>main.o hello.o</code>)
%	Wildcard für Pattern Matching (z.B. <code>%.o: %.c</code>)

Phony Targets

Targets, die keine echten Dateien sind, sondern nur Namen für Aktionen. Sie sollten explizit als `.PHONY` deklariert werden, um Konflikte mit gleichnamigen Dateien zu vermeiden.

- `all`: Baut das gesamte Projekt (oft Standard).
- `clean`: Löscht alle generierten Dateien (Aufräumen).

```
.PHONY: all clean

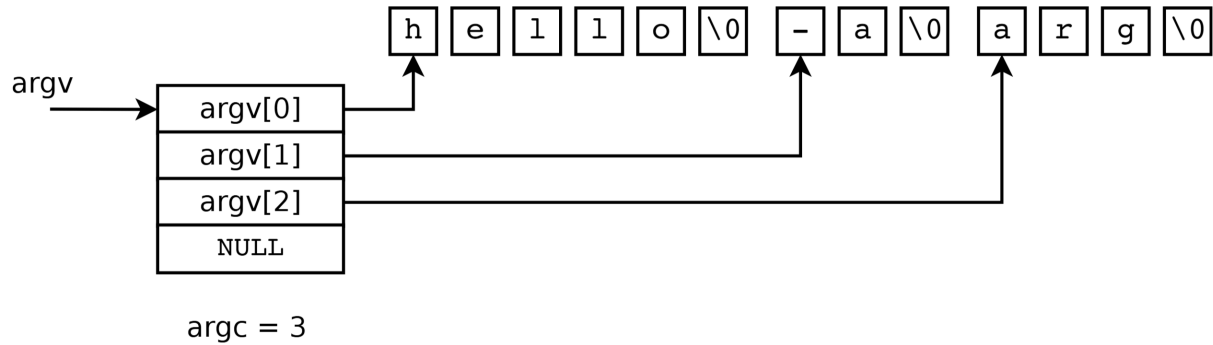
clean:
    rm -rf *.o hello
```

2. Argument Parsing

Wie verarbeitet man Command Line Arguments in C?

Syntax: `int main(int argc, char *argv[])`

- **argc (argument count):** Anzahl der Argumente (inklusive Programmname).
- **argv (argument vector):** Array von Strings.
 - `argv[0]`: Programmname / Pfad.
 - `argv[argc]`: Ist immer `NULL`.



getopt (Library Funktion)

Standard-Weg, um Optionen (Flags) zu parsen (Header: `<unistd.h>`).

```
int getopt(int argc, char *const argv[], const char *optstring);
```

- **optstring:** Definiert gültige Optionen.
 - `"a" ⇒ Option -a (ohne Argument).`
 - `"a:" ⇒ Option -a benötigt ein Argument.`
 - `"a::" ⇒ Option -a hat optionales Argument.`

Globale Variablen von getopt

- `optarg`: Zeigt auf den Wert des Arguments (wenn Option eines braucht).
- `optind`: Index des nächsten zu verarbeitenden Elements in `argv`.

Code-Pattern für getopt

```
int c;
while ((c = getopt(argc, argv, "a:o")) != -1) {
    switch (c) {
        case 'a':
            printf("Option a mit Wert: %s\n", optarg);
            break;
        case 'o':
            flag_o = 1;
            break;
        case '?':
            /* Ungültige Option */
            break;
    }
}
// Nach der Loop: Positionale Argumente verarbeiten
if (optind < argc) {
    printf("Erstes positionales Argument: %s\n", argv[optind]);
}
```

Usage Message

Sollte gedruckt werden, wenn falsche Argumente übergeben werden.

Konvention:

- `[]`: Optional
- `|`: Alternativen
- `...`: Mehrere möglich
- Ausgabe auf `stderr`!

3. Error Handling

Viele C-Funktionen geben bei Fehler `-1` oder `NULL` zurück und setzen die globale Variable `errno`.

🔗 Fehlermeldungen richtig ausgeben

Nutze `strerror(errno)`, um den numerischen Fehlercode in einen lesbaren String umzuwandeln.

```
#include <string.h>
#include <errno.h>

if ((file = fopen("input.txt", "r")) == NULL) {

    fprintf(stderr, "ERROR: fopen failed: %s\n", strerror(errno));

    exit(EXIT_FAILURE);

}
```

Schritte bei fatalen Fehlern:

1. Ressourcen freigeben (soweit möglich).
2. Nachricht auf `stderr`.
3. `exit(EXIT_FAILURE)` aufrufen.

4. Signals

Signale sind Nachrichten vom OS an den Prozess (Inter-Process Communication).

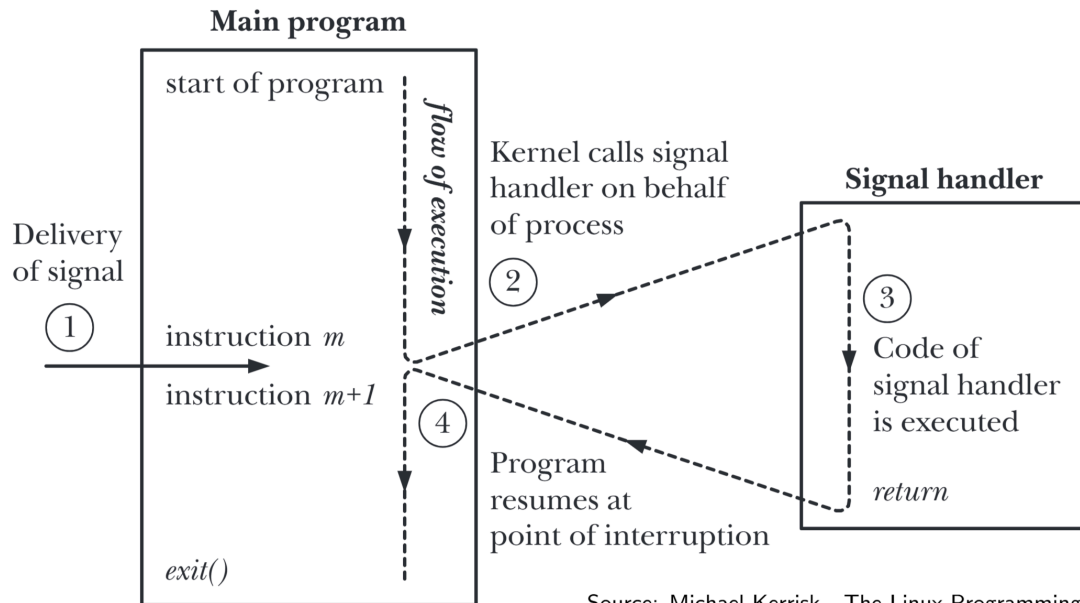
Bekannte Signale:

- `SIGINT`: Interrupt (Ctrl+C).
- `SIGTERM`: Terminierung anfordern.
- `SIGSEGV`: Segmentation Fault (Speicherzugriffsfehler).
- `SIGKILL` / `SIGSTOP`: Können nicht abgefangen/ignoriert werden.

Signal Handling konfigurieren: `sigaction`

Verwende `sigaction` statt der veralteten `signal()` Funktion.

```
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = handle_signal; // Eigene Funktion
sigaction(SIGINT, &sa, NULL);
```



Source: Michael Kerrisk - The Linux Programming Interface, p. 399

Writing Signal Handlers

Ein Signal Handler unterbricht das Hauptprogramm an einer beliebigen Stelle. Das ist gefährlich!

⚠ Async-signal-safe Functions

Innerhalb eines Signal Handlers dürfen nur bestimmte, sichere Funktionen aufgerufen werden.

- **Verboten:** `malloc`, `free`, `printf`, `exit` (wegen globalem State/Locks).
- **Erlaubt:** `write`, `_exit`, Variablen setzen.

Best Practice Pattern

Da man im Handler kaum etwas tun darf, setzt man meistens nur ein Flag.

```
// "volatile" verhindert, dass Compiler die Variable wegoptimiert
// "sig_atomic_t" garantiert atomaren Zugriff (kein halbes Schreiben
möglich)
volatile sig_atomic_t quit = 0;

void handle_signal(int signal) {
    quit = 1;
}

int main() {
    // Setup sigaction ...

    while (!quit) {
        // Main Loop
    }
    // Clean exit
    return 0;
}
```


6. myGrep + 3Coloring Handout

Part I: Files

File Descriptors

Ein Prozess führt eine Tabelle aller offenen Files. Der Index in dieser Tabelle ist der **File Descriptor (fd)**, ein nicht-negativer Integer.

fd	Description	POSIX Name	stdio.h Stream Name
0	Standard input	STDIN_FILENO	stdin
1	Standard output	STDOUT_FILENO	stdout
2	Error output	STDERR_FILENO	stderr

- **Header:** `<unistd.h>` (für POSIX Namen), `<fcntl.h>` (für File Handling Syscalls).

System Calls (Low-Level I/O)

Open

Öffnet oder erstellt eine Datei.

```
int open(const char *pathname, int flags, mode_t mode);
```

- **flags** (Bitwise OR):
 - `O_RDONLY`, `O_WRONLY`, `O_RDWR`: Access Mode (einer muss gesetzt sein).
 - `O_CREAT`: Erstellt Datei, falls nicht existent.
 - `O_EXCL`: Fehler, falls Datei mit `O_CREAT` schon existiert (verhindert Überschreiben).
 - `O_APPEND`: Anhängen statt überschreiben.
- **mode**: Permissions bei Erstellung (z.B. `0600`).
- **Return**: File Descriptor oder `-1` (Error $\Rightarrow$ `errno`).

Read & Write

```
ssize_t read(int fd, void *buf, size_t count);
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

- Gibt die Anzahl der tatsächlich gelesenen/geschriebenen Bytes zurück.
- Bei Fehler: `-1`.
- Bei `read` bedeutet Return `0`: **End of File (EOF)**.

⚠ EINTR Handling

`read()` und `write()` können durch Signale unterbrochen werden.

- Return Wert ist dann `-1`.
- `errno` ist auf `EINTR` gesetzt.
- **Lösung:** Operation wiederholen (`continue` in Loop).

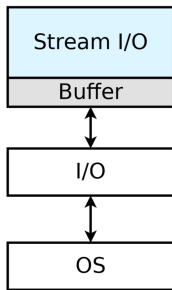
Close

```
int close(int fd);
```

- Wichtig: Ressourcen freigeben. Immer Return-Value checken!
-

Stream I/O (`stdio.h`)

Stream I/O ist ein **gebufferter Wrapper** um die Unix System Calls. Das `FILE` Struct (Stream) enthält den File Descriptor, Buffer-Pointer, Position, EOF- und Error-Flags.



Wichtige Funktionen

- `fopen(path, mode)`: Öffnet File. Modi: `"r"`, `"w"` (truncate!), `"a"`, `"r+"` (update).
- `fdopen(fd, mode)`: Erstellt Stream aus existierendem File Descriptor.
- `fflush(stream)`: Erzwingt das Schreiben der gebufferten Daten auf die Disk.
- `fclose(stream)`: Flushed und schließt den Stream (und den fd).

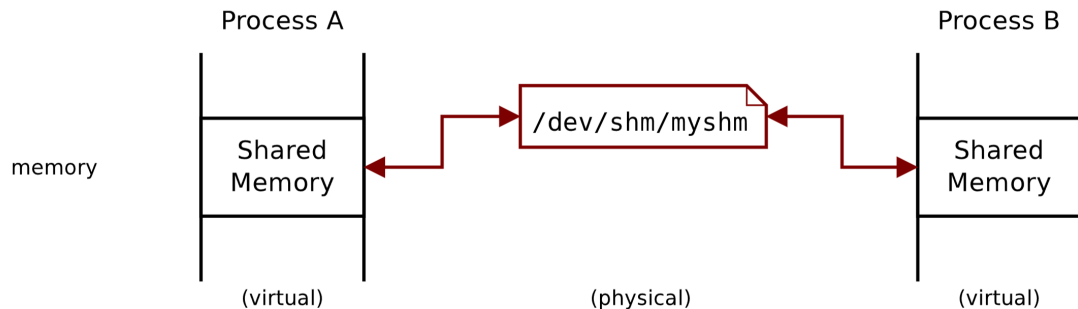
Stream Status prüfen

- `ferror(stream)`: Prüft auf Fehler (0 = kein Fehler).
- `feof(stream)`: Prüft auf End-of-File.
- `fileno(stream)`: Gibt den darunterliegenden File Descriptor zurück.

Part II: Shared Memory (SHM)

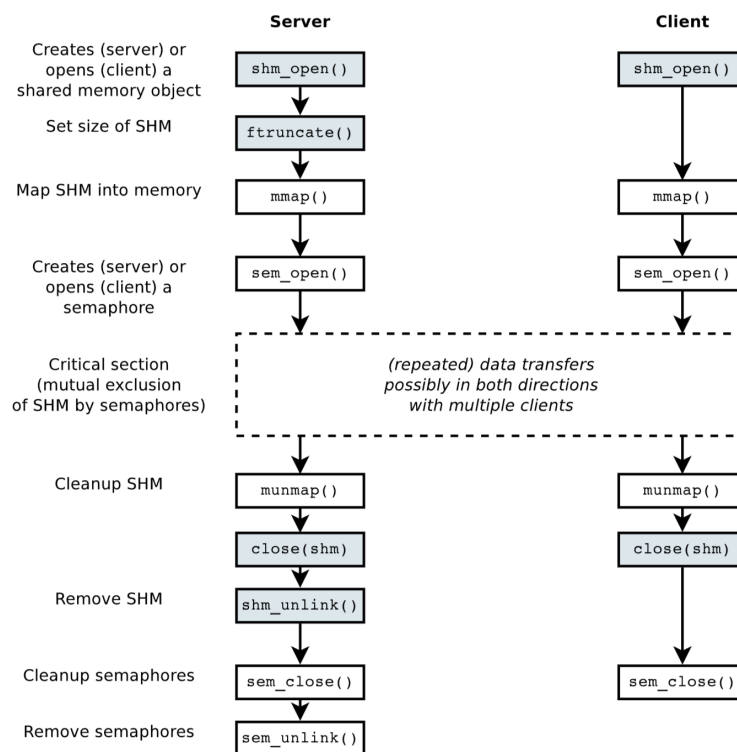
Shared Memory ist ein Speicherbereich, auf den mehrere Prozesse zugreifen können. Er wird in den Adressraum jedes Prozesses gemappt.

- **Vorteil:** Sehr schnelle Inter-Process Communication (IPC).
- **Nachteil:** Prozesse müssen Zugriff synchronisieren (Race Conditions).



Lifecycle eines Shared Memory Objects

Der Ablauf unterscheidet sich leicht zwischen Server (Creator) und Client (User), folgt aber meist diesem Schema:



1. **Create/Open:** `shm_open()`
2. **Set Size:** `ftruncate()` (meist nur durch Creator).
3. **Map:** `mmap()` (macht den Speicher im Prozess verfügbar).
4. **Use:** Lesen/Schreiben wie im normalen RAM.
5. **Unmap:** `munmap()`
6. **Close:** `close()` (File Descriptor schließen).
7. **Remove:** `shm_unlink()` (Name aus dem System entfernen).

Wichtige Funktionen

shm_open

```
int shm_open(const char *name, int oflag, mode_t mode);
```

- Erstellt Object unter `/dev/shm/name`.
- Flags: `O_RDWR`, `O_CREAT`, `O_EXCL`.

ftruncate

```
int ftruncate(int fd, off_t length);
```

- Setzt die Größe des Shared Memory Objekts (Standardgröße nach Creation ist 0).

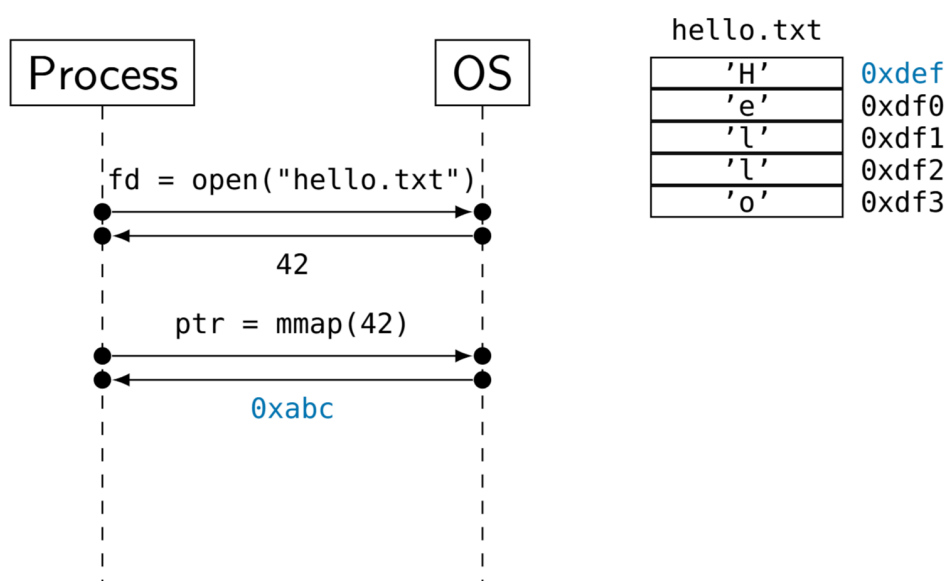
shm_unlink

```
int shm_unlink(const char *name);
```

- Entfernt den Namen. Der Speicher wird erst freigegeben, wenn alle Prozesse ihn unmapped/closed haben.

Memory Mapping (`mmap`)

Mapped eine Datei (oder SHM-Objekt) in den virtuellen Adressraum.



```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- `addr`: Wunschadresse (meist `NULL`, OS entscheidet).

- `prot`: Protection (`PROT_READ | PROT_WRITE`).
- `flags`: `MAP_SHARED` (Änderungen sichtbar für andere) vs. `MAP_PRIVATE` (Copy-on-Write).
- **Return**: Pointer auf den Speicher (`void *`) oder `MAP_FAILED`.

Part III: Semaphores

Mutual Exclusion Problem

Wenn mehrere Threads/Prozesse ungeschützt auf geteilte Ressourcen zugreifen (Critical Section), entstehen Race Conditions (z.B. Memory Leaks durch doppeltes `malloc`).

Semaphore Konzept

Ein Semaphore ist ein atomarer Counter mit zwei Operationen:

- **P(S) / Wait / Down:**
 - Dekrementiert S.
 - Blockiert, wenn $S \leq 0$.
- **V(S) / Post / Signal / Up:**
 - Inkrementiert S.
 - Weckt wartenden Prozess auf.

POSIX Named Semaphores

Ähnlich wie SHM, identifiziert durch Namen (z.B. `/sem_name`). Liegen im `tmpfs`.

Funktionen

- `sem_open(name, flags, mode, value)`: Öffnet/Erstellt Semaphore. `value` ist der Initialwert (nur bei `O_CREAT`).
- `sem_wait(sem)`: Kritische Sektion betreten (P-Operation). Kann durch Signale unterbrochen werden (`EINTR`)!
- `sem_post(sem)`: Kritische Sektion verlassen (V-Operation).
- `sem_close(sem)`: Handle schließen.
- `sem_unlink(name)`: Semaphore entfernen.

Use Cases

1. Mutex (Mutual Exclusion)

Initialwert: 1

Sichert kritische Sektionen.

```
sem_wait(mutex); // S wird 0, andere warten
// Critical Section
sem_post(mutex); // S wird 1, nächster kann rein
```

2. Alternating Execution (Synchronisation)

Zwei Semaphoren, um Reihenfolge zu erzwingen (z.B. Ping-Pong).

- `S1` Init: 1
- `S2` Init: 0

Thread A	Thread B
<code>sem_wait(S1)</code>	<code>sem_wait(S2)</code>
<code>// do work</code>	<code>// do work</code>
<code>sem_post(S2)</code>	<code>sem_post(S1)</code>

Part IV: Circular Buffer Example

Implementierung eines FIFO Buffers in Shared Memory mit Semaphoren.

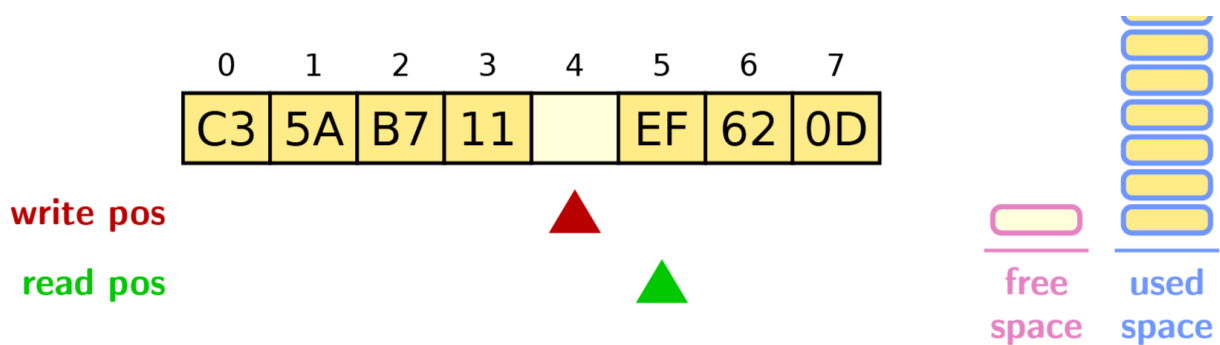
Struktur:

- Array `buf` der Länge `LEN`.
- `wr_pos`: Schreibposition.
- `rd_pos`: Leseposition.

Synchronisation (Producer/Consumer):

Wir brauchen zwei Semaphoren, um Überlauf (Overflow) und Unterlauf (Underflow) zu verhindern.

1. `free`: Initialisiert mit `LEN` (Zählt freien Platz).
2. `used`: Initialisiert mit `0` (Zählt vorhandene Items).



Producer (Write):

```
void write(int val) {
    sem_wait(&free);      // Warten bis Platz frei ist
    buf[wr_pos] = val;    // Schreiben
    wr_pos = (wr_pos + 1) % LEN;
    sem_post(&used);      // Signalisieren: Ein Item mehr da
}
```

Consumer (Read):

```
int read() {
    sem_wait(&used);      // Warten bis Item da ist
    int val = buf[rd_pos]; // Lesen
    rd_pos = (rd_pos + 1) % LEN;
    sem_post(&free);      // Signalisieren: Ein Platz frei geworden
    return val;
}
```


7. myGrep Code-Walkthrough

⚠ Disclaimer

Hier wird myGrep_ue erklärt anhand meiner Lösung. Diese kann Fehler beinhalten oder umständlich geschrieben sein, da ich zum ersten Mal mit C gearbeitet habe! Einige dieser Fehler / Umständlichkeiten werden hier auch behandelt.

📋 Projektübersicht

Das Ziel ist ein vereinfachtes `grep` (Global Regular Expression Print).

Features:

- Suche nach einem Keyword in Dateien oder `stdin`.
- Case-Insensitive Suche (`-i`).
- Ausgabe in eine Datei umleiten (`-o`).
- Modularer Aufbau mit Makefile.

1. Das Build-System: `Makefile`

Das Makefile automatisiert den Kompilierungsprozess. Es sorgt dafür, dass nur geänderte Dateien neu kompiliert werden (Inkrementeller Build).

Variablen & Einstellungen

```
Compiler = gcc
CompilerTags = -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -
D_SVID_SOURCE -D_POSIX_C_SOURCE=200809L -g
```

📄 Warum diese Flags?

- `Compiler = gcc`: Definiert den Compiler. Macht es leicht, später auf `clang` zu wechseln.
- `-std=c99`: Wir nutzen den C99 Standard (erlaubt z.B. `for (int i=0;...)`).
- `-Wall -pedantic`: Aktiviert fast alle Warnungen. **Wichtig für die Prüfung:** Code muss meist ohne Warnungen kompilieren.
- `-D...`: Diese "Defines" aktivieren Feature-Sets in den Header-Dateien (z.B. für `getline` oder `strh` Funktionen, falls benötigt, oder POSIX-Standards).
- `-g`: Fügt Debugging-Informationen hinzu (für `gdb` oder `valgrind`).

Linking (Das finale Programm)

```
mygrep: main.o mygrep.o
$(Compiler) $(CompilerTags) main.o mygrep.o -o mygrep
```

🔗 Linking Erklärung

- **Target:** `mygrep` (Die ausführbare Datei).
- **Dependencies:** `main.o` und `mygrep.o`. Wenn sich eine davon ändert, wird neu gelinkt.
- **Recipe:** Ruft `gcc` auf, nimmt die beiden Objektdateien und erstellt (`-o`) das Binary `mygrep`.

Compiling (Objektdateien erstellen)

```
main.o: main.c mygrep.h
$(Compiler) $(CompilerTags) -c main.c

mygrep.o: mygrep.c mygrep.h
$(Compiler) $(CompilerTags) -c mygrep.c
```

❓ Warum -c?

Das Flag -c sagt dem GCC: "Kompiliere nur, aber linke noch nicht."

- Ergebnis: Eine `.o` Datei (Maschinencode, aber noch keine fertige Executable).
- **Dependency:** Beide hängen von `mygrep.h` ab. Ändert sich der Header, müssen beide neu kompiliert werden.

Cleanup

```
clean:
rm -f *.o mygrep
```

✓ Clean Target

Standard, um aufzuräumen. `rm -f` verhindert Fehler, falls die Dateien gar nicht existieren.

2. Der Header: `mygrep.h`

Die Schnittstelle ("Interface") zwischen `main.c` und `mygrep.c`.

```
#include <stdbool.h>

int grep_process_files(const char *keyword, char **files, int file_count,
bool ignore_case, const char *outfile);

int grep_process_stdin(const char *keyword, bool ignore_case, const char
*outfile);

void tolowerSTR(char *str);
```

🔗 Header Guards fehlen!

In einer Prüfungssituation solltest du unbedingt Header Guards hinzufügen, um doppeltes Einbinden zu verhindern:

```
#ifndef MYGREP_H
#define MYGREP_H
// ... dein code ...
#endif
```

- `const char *`: Zeigt an, dass die Funktion den String (Keyword/Filename) **nicht verändern** wird. Wichtig für "String Literals".
- ``char files`:**` Ein Array von Strings (Zeiger auf Zeiger).

3. Der Einstiegspunkt: `main.c`

Hier passiert das Argument Parsing und die Steuerung des Programmflusses.

Setup & Variablen

```
#include <stdio.h>
// ... weitere includes ...

int main(int argc, char **argv) {
    bool ignore_case = false;
    char *outfile = NULL;
    char *keyword = NULL;
```

Initialisierung

Pointer wie outfile immer mit NULL initialisieren, um später prüfen zu können, ob sie gesetzt wurden (if (outfile != NULL)).

Argument Parsing mit `getopt`

Das ist der Industriestandard zum Parsen von Flags (wie `-i` oder `-o file`).

```
int opt;
// "io:" bedeutet: 'i' ist ein Flag, 'o' ist ein Flag das ein Argument
// benötigt (Doppelpunkt!)
while ((opt = getopt(argc, argv, "io:")) != -1) {
    switch (opt) {
        case 'i':
            ignore_case = true;
            break;
        case 'o':
            outfile = optarg; // optarg ist eine globale Variable von
// getopt
            break;
        default: // Unbekanntes Flag oder fehlendes Argument
            fprintf(stderr, "Usage: %s [-i] [-o outfile] keyword
[file...]\n", argv[0]);
            return EXIT_FAILURE;
    }
}
```

`getopt` Globale Variablen 1

- `optarg`: Wenn ein Flag ein Argument braucht (wie `-o filename`), speichert `getopt` den Zeiger darauf in `optarg`.

- `optind`: Der Index im `argv`-Array, an dem `getopt` gerade steht. Nach der Loop zeigt `optind` auf das erste Argument, das **kein** Flag ist (hier: das Keyword).

Keyword & Dateien verarbeiten

```
// Prüfen, ob nach den Flags noch Argumente übrig sind (mindestens das
// Keyword)
if (optind >= argc) {
    fprintf(stderr, "Usage: ...\n", argv[0]);
    return 1;
}

keyword = argv[optind++]; // Erst Keyword holen, dann Index erhöhen
if (ignore_case) tolowerSTR(keyword); // Keyword direkt normalisieren
```

⚠ Warum tolowerSTR beim Keyword?

Wenn wir Case-Insensitive suchen, ist es am effizientesten, sowohl das Keyword als auch die Zeile in der Datei temporär in Kleinbuchstaben umzuwandeln, um sie zu vergleichen.

Entscheidung: Datei oder Stdin?

```
// Der Rest von argv sind die Dateinamen
char **files = argv + optind; // Pointer-Arithmetik: files zeigt jetzt auf
// den Rest
int file_count = argc - optind;

if (file_count > 0) {
    // Dateien wurden angegeben
    if (grep_process_files(keyword, files, file_count, ignore_case,
outfile) != 0) {
        return EXIT_FAILURE;
    }
} else {
    // Keine Dateien angegeben -> lese von stdin
    if (grep_process_stdin(keyword, ignore_case, outfile) != 0) {
        return EXIT_FAILURE;
    }
}
return EXIT_SUCCESS;
}
```

📄 Pointer Arithmetik bei files

argv ist ein Array von Pointern. argv + optind verschiebt den Startpunkt des Arrays um optind Stellen nach hinten. Das spart uns das Kopieren der Argumente in ein neues Array.

4. Die Logik: `mygrep.c`

Hilfsfunktion: String Lowercase

```
void tolowerSTR(char *str) {
    for (int i = 0; str[i]; i++) { // Läuft solange str[i] != '\0'
        (Nullterminator)
        str[i] = tolower(str[i]);
    }
}
```

Dateien verarbeiten: `grep_process_files`

```
int grep_process_files(const char *keyword, char **files, int file_count,
    bool ignore_case, const char *outfile) {
    int status = 0;

    for (int i = 0; i < file_count; i++) {
        // Debugging Makro (wird nur kompiliert wenn -DDEBUG gesetzt ist)
        #ifdef DEBUG
        printf("Processing file: %s\n", files[i]);
        #endif

        // Ruft die eigentliche Logik pro Datei auf
        if (grep_file(files[i], keyword, ignore_case, outfile) != 0) {
            status = 1; // Merken, dass ein Fehler aufgetreten ist
        }
    }
    return status;
}
```

Die Kern-Logik: `grep_file` (Static)

Diese Funktion ist `static`, weil sie nur innerhalb von `mygrep.c` sichtbar sein soll (Private Helper).

1. Datei öffnen

```
static int grep_file(...) {
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        return 1; // Fehler: Datei existiert nicht oder keine Rechte
    }
}
```

2. Output Datei öffnen (Achtung!)

```

FILE *out = NULL;
if (outfile != NULL) {
    // "a" = append. Wichtig, da wir in der Loop für jede Input-Datei
    // die Output-Datei neu öffnen. "w" würde jedes Mal überschreiben!
    out = fopen(outfile, "a");
    if (out == NULL) {
        fclose(file); // Wichtig: Input File schließen bevor wir returnen!
        return 1;
    }
}
}

```

⚠ Performance Hinweis

In dieser Implementierung wird die Output-Datei für jede Input-Datei geöffnet und geschlossen (fopen / fclose). Bei tausenden Dateien ist das langsam.

Besser: Die Output-Datei einmal in main oder grep_process_files öffnen und das FILE * Handle weiterreichen.

3. Buffer und Loop (Das Herzstück)

```

char line[MAX_LINE_LENGTH]; // Statischer Buffer auf dem Stack

// fgets liest bis zum Newline (\n) oder bis Buffer voll ist
while (fgets(line, sizeof(line), file)) {
    char cmp[MAX_LINE_LENGTH];
    strcpy(cmp, line); // Kopie erstellen, damit wir das Original für den
    Output behalten

    if (ignore_case) {
        tolowerSTR(cmp); // Kopie kleinschreiben
    }

    // strstr prüft ob keyword in cmp enthalten ist
    if (strstr(cmp, keyword)) {
        if (outfile) {
            fprintf(out, "%s", line); // In Datei schreiben
        } else {
            printf("%s", line);      // Auf Konsole schreiben
        }
    }
}
}

```

☰ Buffer Theorie

- Wir brauchen Speicher, um eine Zeile aus der Datei zwischenspeichern.
- Stack-Speicher (wie hier) ist schnell, aber begrenzt.

- `fgets` ist sicher, weil wir die Größe (`sizeof(line)`) mitgeben → Kein Buffer Overflow möglich.

4. Cleanup

```
fclose(file);
if (out != NULL) fclose(out);
return 0;
}
```

✓ Resource Management

Jedes `fopen` braucht ein `fclose`. Vergessene file handles führen zu Memory Leaks oder dazu, dass das OS keine Dateien mehr öffnen kann ("Too many open files").

Stdin Verarbeitung: `grep_process_stdin`

Funktioniert fast identisch zu `grep_file`, mit kleinen Unterschieden:

1. **Input:** Nutzt `stdin` (Standard Input Stream) statt `fopen`.
2. **Output:** Öffnet Output mit `"w"` (Write), da es nur einmal aufgerufen wird und wir eine saubere Datei wollen (oder überschreiben).

```
int grep_process_stdin(...) {
    FILE *output_stream = stdout; // Standardmäßig auf Konsole

    if (outfile != NULL) {
        output_stream = fopen(outfile, "w"); // Hier "w", da einmalig
        if (output_stream == NULL) return 1;
    }

    // ... Buffer Logic gleich wie oben ...

    while (fgets(line, sizeof(line), stdin)) { // Liest von Tastatur/Pipe
        // ... gleiche Suchlogik ...
    }

    // stdout darf man nicht schließen, selbsterstellte Files schon
    if (output_stream != stdout) {
        fclose(output_stream);
    }
    return 0;
}
```

Zusammenfassung für die Prüfung

1. **Modularisierung:** `Makefile` trennt Build-Steps, Header trennt Deklaration von Implementation.
2. **Arguments:** `getopt` ist der Standardweg. Wisse was `argc`, `argv`, `optarg` und `optind` tun.
3. **File I/O:**
 - `fopen("r")` zum Lesen.
 - `fopen("w")` zum Schreiben (löscht Inhalt).
 - `fopen("a")` zum Anhängen.
 - `fgets(buffer, size, stream)` zum zeilenweisen Lesen.
4. **Strings:**
 - `strcpy` zum Kopieren (Vorsicht: Buffer muss groß genug sein!).
 - `strstr` zum Suchen von Substrings.
 - Strings in C sind `char` Arrays mit `\0` am Ende.

8. 3Coloring Code-Walkthrough

Disclaimer

Hier wird die 3coloring_ue erklärt anhand meiner Lösung. Diese kann Fehler beinhalten oder umständlich geschrieben sein, da ich zum ersten Mal mit C gearbeitet habe! Einige dieser Fehler / Umständlichkeiten werden hier auch behandelt.

Projektübersicht

Ein System zur Lösung des 3-Färbungs-Problems von Graphen mittels Randomisierung.

Architektur:

- **Supervisor (Consumer):** Erstellt Shared Memory & Semaphore, liest Lösungen, prüft auf die beste Lösung (0 Kanten entfernen = 3-färbbar).
- **Generator (Producer):** Erzeugt zufällige Färbungen, prüft Konflikte und schreibt "Lösungen" (Liste zu entfernender Kanten) in den Buffer.
- **IPC (Inter-Process Communication):** Shared Memory (Circular Buffer) + POSIX Semaphore.

1. Das Build-System: **Makefile**

Das Makefile linkt hier **spezielle Bibliotheken**, die für Shared Memory und Semaphore unter Linux notwendig sind.

```
Compiler = gcc
CompilerTags = -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -
D_SVID_SOURCE -D_POSIX_C_SOURCE=200809L -g

all: supervisor generator

supervisor: supervisor.o
    $(Compiler) $(CompilerTags) supervisor.o -o supervisor -lrt -pthread

generator: generator.o
    $(Compiler) $(CompilerTags) generator.o -o generator -lrt -pthread
```

Wichtige Linker-Flags

- **-lrt**: "Link Realtime". Wird für `shm_open` und `shm_unlink` benötigt (Shared Memory Objekt Funktionen).
- **-pthread**: Wird für POSIX Semaphoren (`sem_open`, etc.) und Threading-Support benötigt. Ohne diese Flags bekommst du "undefined reference" Fehler beim Linken.

2. Der Header: `coloring.h`

Definiert die Datenstruktur im Shared Memory. Da beide Programme (Supervisor & Generator) darauf zugreifen, muss das Layout **exakt gleich** sein.

```
#define MAX_SOLUTION_EDGES 8
#define BUFFER_SIZE 16

struct Edge { int u; int v; };

struct Solution {
    int count;
    struct Edge edges[MAX_SOLUTION_EDGES];
};

struct CircularBuffer {
    struct Solution solutions[BUFFER_SIZE];
    int writingPosition;
    int readingPosition;
    volatile int shutdown;
};
```

⚡ Warum volatile?

volatile int shutdown;

Das Keyword volatile sagt dem Compiler: "Diese Variable kann sich jederzeit ändern, auch wenn dieses Programm sie nicht anfasst (z.B. durch Hardware oder einen anderen Prozess)."

- **Ohne volatile:** Der Compiler könnte die `while(buffer->shutdown == 0)` Schleife im Generator optimieren und den Wert cachen, sodass er nie merkt, wenn der Supervisor `shutdown` auf 1 setzt.

IPC Namen & Konstanten

```
#define SHM_NAME "/a<MEINE_MATRIKELNUMMER>_shm"
#define SEM_FREE_SPACE "/a<MEINE_MATRIKELNUMMER>_sem_free"
#define SEM_USED_SPACE "/a<MEINE_MATRIKELNUMMER>_sem_used"
#define SEM_WRITE_MUTEX "/a<MEINE_MATRIKELNUMMER>_sem_mutex"
```

i Namenskonvention

Shared Memory und Semaphore Namen sollten unter Linux mit einem / beginnen. Um Kollisionen mit anderen Studenten auf demselben Server zu vermeiden, ist die Matrikelnummer im Namen eine gute Praxis.

3. Der Supervisor: `supervisor.c`

Der Supervisor ist der "Host". Er bereitet die Umgebung vor (Setup), verarbeitet Ergebnisse und räumt am Ende auf.

Setup: Shared Memory

```
// 1. Objekt erstellen/öffnen
int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);

// 2. Größe festlegen
if (ftruncate(shm_fd, sizeof(struct CircularBuffer)) == -1) { ... }

// 3. In den Adressraum mappen
struct CircularBuffer *buffer;
buffer = mmap(NULL, sizeof(struct CircularBuffer), PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
```

❓ Warum ftruncate?

Wenn `shm_open` ein neues Objekt erstellt, hat es die Größe 0 Byte. Wenn wir versuchen würden, darauf zuzugreifen (via `mmap`), gäbe es einen "Bus Error" oder Segfault. `ftruncate` bläst die Datei auf die Größe unseres Structs auf.

Setup: Semaphore

```
// Zählt freien Platz im Buffer (Init: BUFFER_SIZE)
sem_t *sem_free = sem_open(SEM_FREE_SPACE, O_CREAT | O_RDWR, MODE,
BUFFER_SIZE);

// Zählt belegten Platz (Items) im Buffer (Init: 0)
sem_t *sem_used = sem_open(SEM_USED_SPACE, O_CREAT | O_RDWR, MODE, 0);

// Mutex für Schreibzugriff (Init: 1) -> Nur einer darf schreiben
sem_t *sem_write = sem_open(SEM_WRITE_MUTEX, O_CREAT | O_RDWR, MODE, 1);
```

🔗 Semaphoren-Logik

- `O_CREAT`: Erstellt die Semaphore, falls sie nicht existiert.
- Der **letzte Parameter** (z.B. `BUFFER_SIZE`) ist der Initialwert. Dieser wird **nur** verwendet, wenn die Semaphore neu erstellt wird.

Die Consumer Loop

```

while (buffer->shutdown == 0 && ...) {
    // 1. Warten, bis etwas da ist (used > 0)
    if (sem_wait(sem_used) == -1) break;

    // 2. Lesen (Kritische Sektion für Lesen ist hier implizit sicher,
    // da wir nur einen Consumer haben und sem_used uns schützt)
    struct Solution current_solution = buffer->solutions[buffer-
>readingPosition];
    buffer->readingPosition = (buffer->readingPosition + 1) % BUFFER_SIZE;
    // Ringbuffer Logik

    // 3. Platz freigeben (free++)
    sem_post(sem_free);

    // 4. Logik prüfen
    if (current_solution.count == 0) {
        // PERFEKTE LÖSUNG GEFUNDEN
        graph_is_3_colorable = 1;
        buffer->shutdown = 1; // Signal an alle Generatoren
        break;
    }

    // ... beste Lösung tracken ...
}

```

Cleanup (Wichtig!)

```

// 1. Semaphore schließen (für diesen Prozess)
sem_close(sem_used); ...

// 2. Semaphore löschen (Systemweit den Namen entfernen)
sem_unlink(SEM_USED_SPACE); ...

// 3. Memory unmappen
munmap(buffer, sizeof(struct CircularBuffer));

// 4. Shared Memory Objekt löschen
shm_unlink(SHM_NAME);

```

⚠ close vs. unlink

- `close`: "Ich bin fertig mit der Datei/Semaphore".
- `unlink`: "Lösche die Datei/Semaphore aus dem System, sobald der letzte Prozess sie geschlossen hat". Wenn der Supervisor abstürzt ohne unlink, bleibt der Shared Memory im /dev/shm liegen (Müll).

4. Der Generator: `generator.c`

Der Generator ist der "Worker". Er verbindet sich zum existierenden Speicher und produziert Daten.

Argument Parsing (Kanten)

```
// argv: "1-2", "2-3"
char *dash = strchr(edge, '-'); // Pointer auf den Bindestrich
long u = strtol(edge, NULL, 10); // Liest Zahl bis zum '-'
long v = strtol(dash + 1, NULL, 10); // Liest Zahl NACH dem '-'
```

≡ Pointer Arithmetik

Wenn edge auf "10-20" zeigt:

- `edge` → "10-20"
- `dash` → "-20"
- `dash + 1` → "20"

Randomisierung

```
srandom(time(NULL) ^ getpid());
```

❓ Warum XOR mit getpid?

Wenn du das Skript startest: `./supervisor ... & ./generator ... & ./generator ....`

Starten beide Generatoren in der gleichen Sekunde, liefert `time(NULL)` denselben Wert. Beide Generatoren würden exakt dieselben Zufallszahlen würfeln. `getpid()` (Process ID) macht den Seed einzigartig pro Prozess.

Die Producer Loop (Sync)

Hier ist die Reihenfolge der Semaphoren extrem wichtig, um Deadlocks zu vermeiden.

```
while (buffer->shutdown == 0) {
    // ... Lösung berechnen ...

    // 1. Prüfen, ob Platz im Buffer ist (Warten)
    sem_wait(sem_free);

    // 2. Check: Hat Supervisor schon abgeschaltet?
    if (buffer->shutdown == 1) {
        sem_post(sem_free);
        // Wir haben Platz reserviert, aber nutzen ihn nicht ->
        zurückgeben!
    }
}
```

```
        break;
    }

    // 3. Mutex holen für exklusiven Schreibzugriff
    // Nötig, falls wir MEHRERE Generatoren haben
    sem_wait(sem_write);

    // --- KRITISCHE SEKTION START ---
    buffer->solutions[buffer->writingPosition] = current_solution;
    buffer->writingPosition = (buffer->writingPosition + 1) % BUFFER_SIZE;
    // --- KRITISCHE SEKTION ENDE ---

    // 4. Mutex freigeben
    sem_post(sem_write);

    // 5. Bescheid sagen: Ein neues Element ist da!
    sem_post(sem_used);
}
```

☠ Deadlock Gefahr

Würde man `sem_wait(sem_write)` vor `sem_wait(sem_free)` machen:

1. Generator holt Mutex.
2. Buffer ist voll → Generator wartet auf `free`.
3. Supervisor will lesen → braucht (hypothetisch) Mutex oder kann nicht `free` posten, weil er blockiert ist.

⇒ Deadlock.

Regel: Immer Ressourcen von "außen nach innen" anfordern (Erst Platz, dann Schreibrecht).

Zusammenfassung für die Prüfung

1. Shared Memory Lifecycle:

- **Supervisor:** `shm_open(0_CREAT)` → `ftruncate` → `mmap` → ... → `munmap` → `shm_unlink`.
- **Generator:** `shm_open(0)` → `mmap` → ... → `munmap`.

2. Semaphore Pattern (Producer/Consumer):

- `sem_free` (Initialwert N): Schützt vor Überlauf (Overflow).
- `sem_used` (Initialwert 0): Schützt vor Unterlauf (Reading empty buffer).
- `mutex` (Initialwert 1): Schützt die Indizes (`writePos`) bei mehreren Producern.

3. Fehlerbehandlung:

- Immer prüfen ob `mmap == MAP_FAILED` oder `sem_open == SEM_FAILED`.
- Ressourcen aufräumen (`unlink`), sonst bleibt Müll im OS.

4. Circular Buffer:

- Modulo-Operator `%` ist essentiell: `pos = (pos + 1) % SIZE`.